

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
snigdha@Snigdha:~$ cat practical4.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t child1, child2, child3;
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    child1 = fork();

    if (child1 == 0)
    {
        printf("Child 1: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(3);
        printf("Child 1 completed\n");
        return 10;
    }

    child2 = fork();

    if (child2 == 0)
    {
        printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(5);
        printf("Child 2 completed\n");
        return 20;
    }

    child3 = fork();

    if (child3 == 0)
    {
        printf("Child 3: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(2);
        printf("Child 3 completed\n");
        return 30;
    }

    printf("\nParent is waiting for children...\n");

    wait(&status);
    printf("Parent collected one child using wait()\n");

    waitpid(child2, &status, 0);
    printf("Parent collected Child 2 using waitpid()\n");

    waitpid(child1, &status, 0);
    printf("Parent collected Child 1 using waitpid()\n");

    printf("Parent Process Completed\n");

    return 0;
}
```

```

snigdha@Snigdha:~$ nano practical4.c
snigdha@Snigdha:~$ gcc practical4.c -o practical4
snigdha@Snigdha:~$ ./practical4
Parent Process Started
Parent PID: 1294

Child 1: PID = 1295, PPID = 1294

Parent is waiting for children...
Child 2: PID = 1296, PPID = 1294
Child 3: PID = 1297, PPID = 1294
Child 3 completed
Parent collected one child using wait()
Child 1 completed
Child 2 completed
Parent collected Child 2 using waitpid()
Parent collected Child 1 using waitpid()
Parent Process Completed

```

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

snigdha@Snigdha:~$ nano zombie.c
snigdha@Snigdha:~$ cat zombie.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent sleeping for 30 seconds...\n");

        sleep(30);

        printf("Parent completed\n");
    }

    return 0;
}
snigdha@Snigdha:~$ |

```

```
snigdha@Snigdha:~$ nano zombie.c
snigdha@Snigdha:~$ gcc zombie.c -o zombie
snigdha@Snigdha:~$ ./zombie
Parent Process
Child Process
Parent PID: 1558
Child PID: 1559
Parent sleeping for 30 seconds...
Child PID: 1559
Parent completed
```

```
snigdha@Snigdha:~$ ps -el | grep zombie
0 S 1000 1558 349 0 80 0 - 692 hrtime pts/0 00:00:00 zombi
1 Z 1000 1559 1558 0 80 0 - 0 - pts/0 00:00:00 zombi
snigdha@Snigdha:~$ ps -o pid,ppid,state,cmd -p 1558,1559
PID PPID S CMD
1558 349 S ./zombie
1559 1558 Z [zombie] <defunct>
```